S.No: 1	Exp. Name: ***GCD of Two Numbers using Recursion***	**Date: 2026-07-13**

Aim:

Write a C program to find the GCD(Greatest Common Divisor) of two numbers using recursion.

Input Format:

- A single line containing two integers separated by space.

Output Format:

- The output is a single integer that prints the GCD of the given numbers.

Note:

- Refer to visible test cases for better understanding.

Source Code:

Gcd.c

```
#include <stdio.h>

// Function to calculate GCD of two numbers using recursion
int gcd(int a, int b) {

        if(b==0)
                return a;


         return gcd(b,a%b);


}

int main() {
    int num1, num2;
    scanf("%d %d", &num1, &num2);
    int result = gcd(num1, num2);
    printf("GCD: %d", result);
    return 0;
}
```

Execution Results - All test cases have succeeded!

ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

Test Case - 1
User Output
12 18
GCD: 6

Test Case - 2
User Output
7 11
GCD: 1

S.No: 2	Exp. Name: ***Factorial of a number***	**Date:** **2026-07-13**

Aim:

Write a C program to find factorial of a non-negative integer n using recursive function.

Source Code:

fact.c

```
#include <stdio.h>
int Factorial(int N){
        if(N == 0)
                return 1;
        else
                return N * Factorial(N-1);

}
int main() {
        int a,b;
        scanf("%d",&a);
        b=Factorial(a);
        printf("%d",b);

}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1
1

Test Case - 2
User Output
4
24

ID: 2503921314921043

PSNA College of Engineering and Technology

ID: 2503921314921043

2025-2029-CSE(CS)

PSNA College of Engineering and Technology

S.No: 3	Exp. Name: ***Fibonacci Series***	**Date:** **2026-07-13**

Aim:

Write a C program that prints the first N numbers of the Fibonacci series.

Input Format:

- A single line contains an integer representing the value of N.

Output Format:

- The first N numbers of the Fibonacci series are separated by a space.
- Print a space after every number, including the last number.

Source Code:

fibonacciSeries.c

```
#include<stdio.h>
int main()
{
        int n,a=0,b=1,c=1;
        scanf("%d",&n);
        for(int i=0;i<n;i++)
                {
                        printf("%d ",a);
                        a=b;
                        b=c;
                        c=a+b;
                }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
0 1 1 2 3

Test Case - 2

User Output
8
0 1 1 2 3 5 8 13

S.No: 4	Exp. Name: ***Implementation of linear list using array.***	**Date: 2026-07-20**

Aim:

Write a c program to Implement a linear list using array. (perform Insertion and deletion operations)

Source Code:

```
LinearList.c
```


ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
#include<stdio.h>
#define MAX 100
int list[MAX];
int n=0;
void insert()
{
        if(n==MAX)
        {
                printf("List is full\n");
                return;
        }
        int item;
        printf("Element to insert: ");
        scanf("%d",&item);
        list[n++] = item;
}

void deleteElement(){


        int item,i,found=0;
        printf("Element to delete: ");
        scanf("%d",&item);
        do{
                if(list[i]==item)
                {
                        found=1;
                        break;
                }
                i++;
        }while(i<n);
        if(!found){
                printf("Element not found\n");
                return;
        }
        for(;i<n-1;i++)
                list[i]=list[i+1];
        n--;
}
void printList() {
        int i;
        printf("List: ");
        for(i=0;i<n;i++)
                printf("%d ",list[i]);
        printf("\n");
}
```

```
int main()
{
        char ch;
        while(1) {
                printf("I - Insert\nD - Delete\nP - Print\nQ - Quit\n");
                printf("Enter option: ");
                scanf(" %c",&ch);
                switch(ch){
                        case 'I':
                        case 'i':
                                insert();
                                break;
                case 'D':
                case 'd':
                                deleteElement();
                                break;
                case 'P':
                case 'p':
                                printList();
                                break;
                case 'Q':
                case 'q':
                                return 0;
                default:
                        printf("Invalid option!\n");
                }
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
D
Element to delete:
14

Element not found
I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
P
List:
I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
I
Element to insert:
101
I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
I
Element to insert:
102
I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
I
Element to insert:
103
I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
p
List: 101 102 103
I - Insert
D - Delete
P - Print

I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
D
Element to delete:
15
I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
p
List: 14 13
I - Insert
D - Delete
P - Print
Q - Quit
Enter option:
Q

S.No: 5	Exp. Name: ***Implement Single linked list***	**Date: 2026-08-17**

Aim:

Write a program to implement single linked list and its operations.

i. Insert (front and rear end)

ii. Delete (front and rear end)

iii. Search

iv. Reverse

Note: Refer to the visible test cases to strictly match the input and output format.

Source Code:

```
WithHeaderNodes.c
```

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insertFront(struct Node** head, int data) {
        struct Node *newNode = createNode(data);
        newNode->next = *head;
        *head = newNode;
}
void insertRear(struct Node** head, int data) {
    struct Node *newNode = createNode(data);
        if(*head == NULL)
        {
                *head = newNode;
                return;
        }
        struct Node *temp = *head;
        while(temp->next != NULL)
                temp = temp->next;
        temp->next=newNode;
}
void deleteFront(struct Node** head) {
    if(*head == NULL)
        {
                printf("Empty List\n");
                return;
        }
        struct Node *temp = *head;
        *head = (*head)->next;
        free(temp);
}
void deleteRear(struct Node** head) {
    if(*head == NULL)
        {
```

```
                printf("Empty List\n");
                return;
        }
        if((*head)->next == NULL)
        {
                free(*head);
                *head = NULL;
                return;
        }
        struct Node *temp = *head;
        while(temp->next->next != NULL)
                temp = temp->next;
    free(temp->next);
        temp->next=NULL;
}

void reverse(struct Node **head) {
        struct Node *prev = NULL;
        struct Node *curr = *head;
        struct Node *next = NULL;
        while(curr != NULL)
                {
                        next = curr->next;
                        curr->next=prev;
                        prev = curr;
                        curr = next;
                }
        *head = prev;
}
int search(struct Node* head,int data) {
        int pos=0;
        while(head){
                if(head->data==data)
                        return pos;
        head = head->next;
        pos++;
        }
        return -1;
}
void printList(struct Node* head) {
        if(head==NULL){
                printf("NULL\n");
                return;
        }
        while(head){
                printf("%d -> ",head->data);
```

```
                head=head->next;
        }
        printf("NULL\n");
}
int main() {
    // Create a header node
    struct Node* head = NULL;
        int choice, data;
        printf("Menu:\n");
    printf("1. Insert at Front\n");
    printf("2. Insert at Rear\n");
    printf("3. Delete from Front\n");
    printf("4. Delete from Rear\n");
    printf("5. Search\n");
    printf("6. Reverse\n");
    printf("7. Print List\n");
    printf("8. Exit\n");

    while (1) {
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            printf("Enter data: ");
            scanf("%d", &data);
            insertFront(&head, data);
            break;
        case 2:
            printf("Enter data: ");
            scanf("%d", &data);
            insertRear(&head, data);
            break;
        case 3:
            deleteFront(&head);
            break;
        case 4:
            deleteRear(&head);
            break;
        case 5:
            printf("Enter element to search: ");
            scanf("%d", &data);
            int pos = search(head, data);
            if (pos != -1)
                printf("Element found\n");
            else
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 16

```
                printf("Element not found\n");
            break;
        case 6:
            reverse(&head);
            printf("List reversed\n");
            break;
        case 7:
            printf("Linked List: ");
            printList(head);
            break;
        case 8:
            printf("Exiting program\n");
            exit(0);
        default:
            printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Menu:
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Search
6. Reverse
7. Print List
8. Exit
Enter your choice:
5
Enter element to search:
6
Element not found
Enter your choice:
10

Enter your choice:
6
List reversed
Enter your choice:
7
Linked List: 2 -> 1 -> NULL
Enter your choice:
8
Exiting program

Test Case - 3
User Output
Menu:
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Search
6. Reverse
7. Print List
8. Exit
Enter your choice:
1
Enter data:
10
Enter your choice:
1
Enter data:
20
Enter your choice:
1
Enter data:
30
Enter your choice:
7
Linked List: 30 -> 20 -> 10 -> NULL
Enter your choice:
4
Enter your choice:
7

S.No: 6	Exp. Name: ***Implementation of Stack using Linked List***	**Date: 2026-08-17**

Aim:

Write a C program that implements a stack using linked list. The program should have **push()**, **pop()**, **display(), isEmpty()** and **peek()** functions that uses switch cases and works on the choice of the user.

Source Code:

```
StackListPopPeek.c
```


ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
#include <stdio.h>
#include <stdlib.h>

struct stack {
        int data;
        struct stack *next;
};

typedef struct stack *stk;
stk top = NULL;


void pop() {
        if(top==NULL){
                printf("Stack is underflow.\n");
                return;
        }
        stk temp = top;
        printf("Popped value = %d\n",top->data);
        top=top->next;
        free(temp);
}

void peek() {
        if(top == NULL){
                printf("Stack is underflow.\n");
                return;
        }
        printf("Peek value = %d\n",top->data);
}

void isEmpty() {
        if(top==NULL)
                printf("Stack is empty.\n");
        else
                printf("Stack is not empty.\n");
}

void push(int x) {
        stk newNode = (stk)malloc(sizeof(struct stack));
        if(newNode == NULL){
                printf("Stack is overflow\n");
                return;
        }
        newNode->data=x;
        newNode->next=top;
```

```
        top=newNode;
        printf("Successfully pushed.\n");
}

void display() {
        if(top == NULL){
                printf("Stack is empty.\n");
                return;
        }
        stk temp=top;
        printf("Elements of the stack are : ");
        while(temp!=NULL){
                printf("%d ",temp->data);
                temp=temp->next;
        }
        printf("\n");

}

int main() {
        int op, x;
        while(1) {
        printf("1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit\n");
                printf("Enter your option : ");
                scanf("%d",&op);
                switch(op) {
                        case 1:
                                printf("Enter element : ");
                                scanf("%d",&x);
                                push(x);
                                break;
                        case 2:
                                pop();
                                break;
                        case 3:
                                display();
                                break;
                        case 4:
                                isEmpty();
                                break;
                        case 5:
                                peek();
                                break;
                        case 6:
                                exit(0);
                }
```

```
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
1
Enter element :
10
Successfully pushed.
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
1
Enter element :
20
Successfully pushed.
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
1
Enter element :
30
Successfully pushed.
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
2
Popped value = 30
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
3
Elements of the stack are : 20 10
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
4
Stack is not empty.
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit

```
Enter element :
45
Successfully pushed.
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
1
Enter element :
67
Successfully pushed.
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
3
Elements of the stack are : 67 45
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
5
Peek value = 67
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
3
Elements of the stack are : 67 45
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
2
Popped value = 67
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
2
Popped value = 45
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
2
Stack is underflow.
1.Push 2.Pop 3.Display 4.Is Empty 5.Peek 6.Exit
Enter your option :
6
```

S.No: 7	Exp. Name: ***Implementation of Queue using Linked List***	**Date: 2026-08-17**

Aim:

Write a C program that implements a **Queue** using linked list. The program should have **enqueue()**, **dequeue(), size()** and **isEmpty()**, **Display()** functions that uses switch cases and works on the choice of the user.

Source Code:

```
QueueListDequeueSize.c
```


ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
#include <stdlib.h>
#include <stdio.h>

struct queue {
        int data;
        struct queue *next;
};

typedef struct queue *Q;
Q front = NULL, rear = NULL;

void enqueue(int element) {
        Q newNode = (Q)malloc(sizeof(struct queue));
        newNode->data=element;
        newNode->next=NULL;
        if(rear == NULL)
                front = rear = newNode;
        else{
                rear->next=newNode;
                rear=newNode;
        }
        printf("Successfully inserted.\n");
}

void dequeue() {
        if(front == NULL){
                printf("Queue is underflow.\n");
                return;
        }
        Q temp = front;
        printf("Deleted value = %d\n",temp->data);
        front = front->next;
        if(front==NULL)
                rear = NULL;
        free(temp);
}
void display() {
        if(front==NULL){
                printf("Queue is empty.\n");
                return;
        }
        Q temp = front;
        printf("Elements in the queue : ");
        while(temp!=NULL){
                printf("%d ",temp->data);
                temp=temp->next;
```

```
        }
        printf("\n");
}

void size() {
        int count = 0;
        Q temp = front;
        while(temp!=NULL){
                count++;
                temp=temp->next;
        }
        printf("Queue size : %d\n",count);
}

void isEmpty() {
        if(front==NULL)
                printf("Queue is empty.\n");
        else
                printf("Queue is not empty.\n");
}

int main() {
        int op, x;
        while(1) {
                printf("1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size
6.Exit\n");
                printf("Enter your option : ");
                scanf("%d",&op);
                switch(op) {
                        case 1:
                                printf("Enter element : ");
                                scanf("%d",&x);
                                enqueue(x);
                                break;
                        case 2:
                                dequeue();
                                break;
                        case 3:
                                display();
                                break;
                        case 4:
                                isEmpty();
                                break;
                        case 5:
                                size();
                                break;
```

```
                    case 6: exit(0);
                }
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
1
Enter element :
12
Successfully inserted.
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
1
Enter element :
14
Successfully inserted.
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
1
Enter element :
15
Successfully inserted.
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
3
Elements in the queue : 12 14 15
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
5
Queue size : 3
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
4

```
Enter your option :
3
Queue is empty.
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
5
Queue size : 0
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
1
Enter element :
23
Successfully inserted.
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
1
Enter element :
45
Successfully inserted.
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
3
Elements in the queue : 23 45
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
2
Deleted value = 23
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
5
Queue size : 1
1.Enqueue 2.Dequeue 3.Display 4.Is empty 5.Size 6.Exit
Enter your option :
6
```

S.No: 8	Exp. Name: ***Single Linked List operations***	**Date: 2026-10-19**

Aim:

Design, Develop and Implement a menu driven Program in C for the following operations on Singly Linked List (SLL) of Student Data with the fields: USN, Name, Branch, Sem, PhNo.

1. Create a SLL of N Students Data by using front insertion.
2. Display the status of SLL and count the number of nodes in it
3. Insertion at End of SLL
4. Deletion at End of SLL
5. Perform Insertion / Deletion at Front of SLL(Demonstration of stack)
6. Exit.

Source Code:

sll.c

ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
#include<stdio.h>
#include<stdlib.h>

struct node
{   char usn[25], name[25], branch[25];
    int sem;
    long int phone;
    struct node *link;
};
typedef struct node * NODE;
NODE start = NULL;
int count = 0;

NODE create()
{
        NODE temp;
        long long *ph;
        temp = (NODE)malloc(sizeof(struct node) + sizeof(long long));
        if(temp == NULL)
        {
                printf("Memory allocation fails\n");
                exit(0);
        }
        ph = (long long *)((char *)temp + sizeof(struct node));
        printf("Enter usn, Name, Branch, sem, PhoneNo of student: ");
        scanf("%s %s %s %d %lld",
                temp->usn,
                temp->name,
                temp->branch,
                &temp->sem,
                ph);
        temp->phone=0;
        temp->link = NULL;
        return temp;
}
NODE insertfront()
{
        NODE temp;
        temp = create();
        temp->link = start;
        start = temp;

        count++;
        return start;
}
```

```
NODE deletefront()
{
        NODE temp;
        if (start == NULL)
        {
                printf("Linked list is empty\n");
                return start;
        }
        temp = start;
        start = start->link;
        printf("The Student node with usn: %s is deleted\n", temp->usn);
        free(temp);
        count--;
        return start;
}

NODE insertend()
{
        NODE temp, cur;
        temp = create();
        if (start == NULL)
        {
                start = temp;
        }
        else
        {
                cur = start;
                while (cur->link != NULL)
                        {
                                cur = cur->link;
                        }
                cur->link = temp;
        }
        count++;
        return start;
}

NODE deleteend()
{
        NODE cur, prev;
        if (start == NULL)
        {
                printf("Linked List is empty\n");
                return start;
        }
        if (start->link == NULL)
```

```
        {
                printf("The student node with usn: %s is
deleted\n",start->usn);
                free(start);
                start = NULL;
                count--;
                return start;
        }
        cur = start;

        while (cur->link != NULL)
                {
                        prev = cur;
                        cur = cur->link;
                }
        printf("The student node with the usn: %s is deleted\n", cur-
>usn);
        prev->link=NULL;
        free(cur);
                count--;
        return start;
}




void display()
{   NODE cur;
         long long ph;
        if(start == NULL)
        {       printf("---No elements to display---\n");
                return;
        }

        printf("The details of the students: \n");
        cur = start;
        int index=1;
        while (cur != NULL)
                {
                        ph= *(long long *)((char *)cur + sizeof(struct
node));



                        printf("--%d-- USN:%s| Name:%s| Branch:%s| Sem:
%d| Ph:%lld|\n",
                                index,
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 37

```
                                cur->usn,
                                cur->name,
                                cur->branch,
                                cur->sem,ph);
                        cur = cur->link;
                        index++;
                }
        printf("No of student nodes is %d\n", count);
}
void stackdemo()
{
        int ch;
        while(1)
                {
                        printf("1: Push operation\n");
                        printf("2: Pop operation\n");
                        printf("3: Display\n");
                        printf("4: Exit\n");

                        printf("Enter your choice for stack demo: ");
                        scanf("%d",&ch);

                        switch(ch)
                                {
                                        case 1:
                                                start = insertfront();
                                                break;
                                        case 2:
                                                start = deletefront();
                                                break;
                                        case 3:
                                                display();
                                                break;
                                        case 4:
                                                return;
                                        default:
                                        printf("Please enter valid
choice\n");
                                }
                }
}

int main()
{   int ch,i,n;
    while(1)
    {   printf("1: Create SLL of Student Nodes");
```

```
        printf("\n2: DisplayStatus");
        printf("\n3: InsertAtEnd");
        printf("\n4: DeleteAtEnd");
        printf("\n5: Stack Demo using SLL(Insertion and Deletion at
Front)");
        printf("\n6: Exit");
        printf("\nEnter your choice: ");
        scanf("%d",&ch);

        switch(ch)
        {   case 1 : printf("Enter the no of students: ");
                     scanf("%d",&n);
                     for(i=1;i<=n;i++)
                        start = insertfront();
                     break;
            case 2 : display();
                     break;
            case 3 : start = insertend();
                     break;
            case 4 : start = deleteend();
                     break;
            case 5 : stackdemo();
                     break;
            case 6 : exit(0);
            default: printf("Please enter valid choice\n");
        }
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1: Create SLL of Student Nodes
2: DisplayStatus
3: InsertAtEnd
4: DeleteAtEnd
5: Stack Demo using SLL(Insertion and Deletion at Front)
6: Exit
Enter your choice:
1

```
Enter the no of students:
3
Enter usn, Name, Branch, sem, PhoneNo of student:
1328 Prasanth CSE 3 1234567890
Enter usn, Name, Branch, sem, PhoneNo of student:
1207 Sashank CSE 4 0987654321
Enter usn, Name, Branch, sem, PhoneNo of student:
1209 Akhil ECE 3 6574839201
1: Create SLL of Student Nodes
2: DisplayStatus
3: InsertAtEnd
4: DeleteAtEnd
5: Stack Demo using SLL(Insertion and Deletion at Front)
6: Exit
Enter your choice:
2
The details of the students:
--1-- USN:1209| Name:Akhil| Branch:ECE| Sem:3| Ph:6574839201|
--2-- USN:1207| Name:Sashank| Branch:CSE| Sem:4| Ph:987654321|
--3-- USN:1328| Name:Prasanth| Branch:CSE| Sem:3| Ph:1234567890|
No of student nodes is 3
1: Create SLL of Student Nodes
2: DisplayStatus
3: InsertAtEnd
4: DeleteAtEnd
5: Stack Demo using SLL(Insertion and Deletion at Front)
6: Exit
Enter your choice:
4
The student node with the usn: 1328 is deleted
1: Create SLL of Student Nodes
2: DisplayStatus
3: InsertAtEnd
4: DeleteAtEnd
5: Stack Demo using SLL(Insertion and Deletion at Front)
6: Exit
Enter your choice:
2
The details of the students:
--1-- USN:1209| Name:Akhil| Branch:ECE| Sem:3| Ph:6574839201|
--2-- USN:1207| Name:Sashank| Branch:CSE| Sem:4| Ph:987654321|
No of student nodes is 2
```

5: Stack Demo using SLL(Insertion and Deletion at Front)
6: Exit
Enter your choice:
3
Enter usn, Name, Branch, sem, PhoneNo of student:
0956 Tanjiro Anime 2 1234
1: Create SLL of Student Nodes
2: DisplayStatus
3: InsertAtEnd
4: DeleteAtEnd
5: Stack Demo using SLL(Insertion and Deletion at Front)
6: Exit
Enter your choice:
2
The details of the students:
--1-- USN:0956| Name:Tanjiro| Branch:Anime| Sem:2| Ph:1234|
No of student nodes is 1
1: Create SLL of Student Nodes
2: DisplayStatus
3: InsertAtEnd
4: DeleteAtEnd
5: Stack Demo using SLL(Insertion and Deletion at Front)
6: Exit
Enter your choice:
6

S.No: 9	Exp. Name: ***Check for the balanced parenthesis using a stack***	**Date: 2026-09-27**

Aim:

Write a C program to check if the parentheses in a given expression are balanced. The program will recognise the following types of brackets: (), {}, and [].

Input Format:

- The user is prompted to enter an expression containing various types of parentheses. The input is a single line string.

Output Format:

The program will display one of the following messages:

- "balanced" if all parentheses are properly matched and nested.
- "not balanced" if there are any mismatches or unclosed parentheses.

Source Code:

```
BalancedParenthesis.c
```


ID: 250392131492104 3
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#define STACK_MAX_SIZE 30
char arr[STACK_MAX_SIZE];
int top = -1;
void push(char element) {
        if (top < STACK_MAX_SIZE - 1) {
                arr[++top] = element;
        }
}

char pop() {
        if (top >= 0) {
                return arr[top--];
        }
        return'\0';
}

int isempty() {
        return top == -1;
}

int isBalanced(char exp[]) {
        int i;
        char popped;

        for (i = 0; i < strlen(exp); i++) {
                if (exp[i] == '(' || exp[i] =='{' || exp[i] == '[') {
                        push(exp[i]);
                }
                else if (exp[i] == ')' || exp[i] == '}' || exp[i] ==']')
{
                        if(isempty()) {
                                return 0;
                        }
                        popped = pop();
                        if ((exp[i] == ')' && popped != '(') ||
                        (exp[i] =='}' && popped != '{') ||
                        (exp[i] == ']' && popped != '[')){
                                return 0;
                        }
                }
        }
        return isempty() ? 1:0;
```

```
}

void main() {
        char ch[80], temp;
        printf("Enter an expression: ");
        scanf("%s", ch);
        if(isBalanced(ch) == 1) {
                printf("balanced\n");
        } else {
                printf("not balanced\n");
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter an expression:
1+2*3+(3+4)
balanced

Test Case - 2
User Output
Enter an expression:
1+2*(3+([4+5])
not balanced

Test Case - 3
User Output
Enter an expression:
{[]}
balanced

S.No: 10	Exp. Name: ***Round Robin Algorithm using Queue***	**Date: 2026-09-27**

Aim:

Write a C program to simulate the Round Robin CPU Scheduling algorithm using a queue.

You are provided with the **main()** function, the required structures, and an incomplete implementation of the program. Your task is to complete the implementation of the following functions:

- **void enqueue(PROCESS p)** - Implement the logic to enqueue a process.
- **PROCESS dequeue()** - Implement the logic to dequeue a process.
- **int isEmpty()** - Implement the logic to check if the queue is empty.
- **void inputProcesses(PROCESS processes, int NOP)** - Implement the logic to take user input for the processes.
- **void roundRobinScheduling(PROCESS processes, int NOP, int time_quantum)** - Implement the Round Robin scheduling logic.

Input Format:

- The program first prompts the user to enter:

```
no of process:
```

- Read an integer NOP, representing the number of processes.
- For each process, the program prompts:

```
Process i name:
```

- Read the name of the process as a string.

```
Process i time:
```

- Read an integer representing the burst time of the process.
- After all process details have been entered, the program prompts:

```
Time quantum:
```

- Read an integer representing the time quantum for Round Robin scheduling.

Output Format

Display the processes in the order of their completion as:

```
Printing processes in the order of their completion
Proc_NameCPU_TimeWait_TimeTurnAround_time
```

For each completed process, print its:

- Process name
- CPU time
- Waiting time
- Turnaround time

Finally, print:

```
Total wait time: <total_wait_time>
Average wait time: <average_wait_time>
Total turn around time: <total_turnaround_time>
Average turn around time: <average_turnaround_time>
```

- where the average waiting time and average turnaround time are displayed with

two digits after the decimal point.

Note:

- Assume that all processes arrive at time 0.
- Each process consists of a process name and its burst time.
- Use the given queue implementation to maintain the ready queue.
- Print the processes in the order in which they complete execution.
- The waiting time for a process is calculated as:

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

Source Code:

```
roundrobin.c
```

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

struct process {
    char name[50];
    int burst_time;
    int cpu_time;
    int wait_time;
    int rem_time;
    int turnaround_time;
};

typedef struct process* PROCESS;

struct queue {
    PROCESS proc;
    struct queue *next;
};

typedef struct queue *Q;
Q front = NULL, rear = NULL;

void enqueue(PROCESS p) {
        Q newNode = (Q)malloc(sizeof(struct queue));
        newNode->proc = p;
        newNode->next = NULL;
        if (rear == NULL) {
                front = rear = newNode;
        }else {
                rear->next = newNode;
                rear = newNode;
        }
}

PROCESS dequeue() {
        if (front == NULL) {
                return NULL;
        }
        Q temp = front;
        PROCESS p = temp->proc;
        front = front->next;
        if (front == NULL) {
                rear = NULL;
        }
        free(temp);
```

```
        return p;
}

int isEmpty() {
        return (front == NULL);
}

void inputProcesses(PROCESS processes, int NOP) {
        for (int i = 0; i < NOP; i++) {
                printf("Process %d name: ", i + 1);
                scanf("%s", processes[i].name);
                printf("Process %d time: ", i + 1);
                scanf("%d", &processes[i].cpu_time);
                processes[i].rem_time = processes[i].cpu_time;
                processes[i].wait_time = 0;
                processes[i].turnaround_time = 0;
        }
}

void roundRobinScheduling(PROCESS processes, int NOP, int time_quantum)
{
    int current_time = 0;
        int complete = 0;
        for (int i = 0; i < NOP; i++){
                enqueue(&processes[i]);
        }

        printf("Printing processes in the order of their completion\n");
    printf("Proc_Name\tCPU_Time\tWait_Time\tTurnAround_time\n");
        while (complete < NOP) {
                if (isEmpty()) {
                        break;
                }
                PROCESS curr = dequeue();
                int exec_time = (curr->rem_time < time_quantum) ? curr-
>rem_time : time_quantum;
                current_time += exec_time;
                curr->rem_time -= exec_time;
                if (curr->rem_time == 0) {
                        complete++;
                        curr->turnaround_time = current_time;
                curr->wait_time = curr->turnaround_time - curr-
>cpu_time;
        printf("%9s\t%8d\t%9d\t%15d\n",
                        curr->name,
                        curr->cpu_time,
```

```
                        curr->wait_time,
                        curr->turnaround_time);
        }else {
                enqueue(curr);
        }
}
int total_wait_time = 0, total_turnaround_time = 0;
for (int i = 0; i < NOP; i++) {
        total_wait_time += processes[i].wait_time;
        total_turnaround_time += processes[i].turnaround_time;
}
    printf("Total wait time: %d\n",  total_wait_time);
    printf("Average wait time: %.2f\n", (float)total_wait_time / NOP);
    printf("Total turn around time: %d\n", total_turnaround_time);
    printf("Average turn around time: %.2f\n",
(float)total_turnaround_time / NOP);
}

int main() {
    int NOP, time_quantum;
    printf("no of process: ");
    scanf("%d", &NOP);

    PROCESS processes = (PROCESS)malloc(NOP * sizeof(struct process));
    inputProcesses(processes, NOP);

    printf("Time quantum: ");
    scanf("%d", &time_quantum);
    roundRobinScheduling(processes, NOP, time_quantum);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
no of process:
3
Process 1 name:
Calc
Process 1 time:

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 53

5

Process 2 name:

Notepad

Process 2 time:

9

Process 3 name:

Wordpad

Process 3 time:

23

Time quantum:

3

Printing processes in the order of their completion

Proc_Name	CPU_Time	Wait_Time	TurnAround_time
Calc	5	6	11
Notepad	9	11	20
Wordpad	23	14	37

Total wait time: 31

Average wait time: 10.33

Total turn around time: 68

Average turn around time: 22.67

Test Case - 2

User Output

no of process:

3

Process 1 name:

turboc

Process 1 time:

43

Process 2 name:

javac

Process 2 time:

54

Process 3 name:

python

Process 3 time:

43

Time quantum:

4

Printing processes in the order of their completion			
Proc_Name	CPU_Time	Wait_Time	TurnAround_time
turboc	43	80	123
python	43	87	130
javac	54	86	140
Total wait time: 253			
Average wait time: 84.33			
Total turn around time: 393			
Average turn around time: 131.00			

S.No: 11	Exp. Name: ***C program to Sort given elements using Quick sort***	**Date: 2026-09-27**

Aim:

Write a C program to sort an array of integers in ascending order using the Quick Sort algorithm.

Input Format:

- First, program will prompt the user to enter:

```
Enter array size :
```

- Then read an integer n, representing the size of the array.
- Next, the program prompts the user with:

```
Enter n elements :
```

- where n is the array size.
- Then read n space-separated integers representing the elements of the array.

Output Format:

- Print the array before sorting as:

```
Before sorting the elements are : <elements separated by space>
```

- After sorting the array with Quick Sort, print:

```
After sorting the elements are : <elements separated by space>
```

Note:

- The **QuickSortMain.c** file contains the **main()** function and the driver code.
- You are required to complete the implementation of the required functions in the **QuickSortFunctions.c** file.
- Each output line should end with a newline character.
- Refer to the visible test cases for better understanding regarding the Input and Output Formats

Source Code:

```
QuickSortMain.c
```

Page No: 56
ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```c
#include <stdio.h>
#include "QuickSortFunctions.c"
void main() {
        int arr[15], i, n;
        printf("Enter array size : ");
        scanf("%d", &n);
        printf("Enter %d elements : ", n);
        for (i = 0; i < n; i++) {
                scanf("%d", &arr[i]);
        }
        printf("Before sorting the elements are : ");
        display(arr, n);
        quickSort(arr, 0, n - 1);
        printf("After sorting the elements are : ");
        display(arr, n);
}
```

QuickSortFunctions.c

```
// Function to display the elements of the array.
// Prints all elements from index 0 to n-1 separated by spaces.
void display(int arr[15], int n) {
        int i;
        for(i = 0; i < n; i++){
                printf("%d ", arr[i]);
        }
        printf("\n");
}

// Function to partition the array for QuickSort algorithm.
// Returns the index position of the pivot element after partition.
int partition(int arr[15], int lb, int ub) {
        int pivot = arr[lb];
        int start = lb;
        int end = ub;
        int temp;
        while(start < end){
                while(arr[start] <= pivot && start < ub)
                        start++;
                while(arr[end] > pivot)
                        end--;
                if(start < end){
                        temp = arr[start];
                        arr[start] = arr[end];
                        arr[end] = temp;
                }
        }
        temp = arr[lb];
        arr[lb] = arr[end];
        arr[end] = temp;

        return end;
}

// QuickSort function to sort the array in ascending order.
// Recursively sorts the elements of the array using the QuickSort
algorithm.
void quickSort(int arr[15], int low, int high) {
        int loc;
        if(low < high){
                loc = partition(arr, low, high);
                quickSort(arr, low, loc - 1);
                quickSort(arr, loc + 1, high);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter array size :
5
Enter 5 elements :
34 67 12 45 22
Before sorting the elements are : 34 67 12 45 22
After sorting the elements are : 12 22 34 45 67

Test Case - 2
User Output
Enter array size :
8
Enter 8 elements :
77 55 22 44 99 33 11 66
Before sorting the elements are : 77 55 22 44 99 33 11 66
After sorting the elements are : 11 22 33 44 55 66 77 99

S.No: 12	Exp. Name: ***Sort given elements using Selection sort smallest element method***	**Date: 2026-09-27**

Aim:

Write a C program that reads n integer elements into an array, prints the array before sorting, sorts the array in ascending order using the selection sort algorithm, and then prints the array after sorting.

The program should prompt the user for input exactly as shown below and print all outputs with precise formatting.

Input Format:

- First, program will prompt the user to enter `Enter value of n :`
- Then read an integer n, representing the number of elements in the array.
- For each element i from 0 to n-1, program will prompt the user to enter: `Enter element for a[i] :`
- where i is the index of the element.
- Read the integer element for the array index i.

Output Format:

- After reading all elements and before sorting, print: `Before sorting the elements in the array are`
- Then print each element on a separate line in the form: `Value of a[i] = <element>`
- where i is the array index and <element> is the value at that index.
- After sorting with selection sort, print: `After sorting the elements in the array are`
- Then print the sorted elements similarly on separate lines: `Value of a[i] = <element>`
- for each i in the sorted array.

Note:

- Refer to the visible test cases for better understanding regarding Input and Output Formats.

Source Code:

```
Program507.c
```

```c
#include<stdio.h>
// Function prototype for selection sort
void selectionSort(int arr[], int n) {
int i, j, min, temp;
for(i = 0; i < n - 1; i++){
        min = i;
        for(j = i + 1; j < n; j++){
                if(arr[j] < arr[min]){
                        min = j;
                }
        }
        if(min != i){
                temp = arr[i];
                arr[i] = arr[min];
                arr[min] = temp;
        }
}
}
int main() {
    int a[20], i, n;

    // Getting the number of elements from the user
    printf("Enter value of n : ");
    scanf("%d", &n);

    // Getting elements from the user and storing them in the array
    for (i = 0; i < n; i++) {
        printf("Enter element for a[%d] : ", i);
        scanf("%d", &a[i]);
    }

    // Displaying the elements before sorting
    printf("Before sorting the elements in the array are\n");
    for (i = 0; i < n; i++) {
        printf("Value of a[%d] = %d\n", i, a[i]);
    }

    // Calling the selection sort function
    selectionSort(a, n);

    // Displaying the elements after sorting
    printf("After sorting the elements in the array are\n");
    for (i = 0; i < n; i++) {
        printf("Value of a[%d] = %d\n", i, a[i]);
    }
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 61

```
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter value of n :
5
Enter element for a[0] :
34
Enter element for a[1] :
78
Enter element for a[2] :
56
Enter element for a[3] :
12
Enter element for a[4] :
48
Before sorting the elements in the array are
Value of a[0] = 34
Value of a[1] = 78
Value of a[2] = 56
Value of a[3] = 12
Value of a[4] = 48
After sorting the elements in the array are
Value of a[0] = 12
Value of a[1] = 34
Value of a[2] = 48
Value of a[3] = 56
Value of a[4] = 78

S.No: 13	Exp. Name: ***Search an Element using Binary Search Process***	**Date: 2026-09-27**

Aim:

Write a program to search a key element in the given array of elements using binary search.

Functions:

void bubbleSort(int a[], int n) :

- Sorts the array a of size n in ascending order using the bubble sort algorithm.

int binarySearch(int a[], int n, int key):

- Performs binary search for key in the sorted array a of size n. Returns the index of the key if found, or -1 if not found.

void inputArray(int a[], int n):

- Inputs n array elements from the user.

void printArray(int a[], int n):

- Prints the elements of the array with index and value

Note:

- Refer to the visible test cases for better understanding regarding Input/Output Format.

Source Code:

```
Program510.c
```


ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```c
#include <stdio.h>

// Function to perform bubble sort
void bubbleSort(int a[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

// Function to perform binary search
int binarySearch(int a[], int n, int key) {

        int low = 0, high = n - 1, mid;
        while (low <= high)
        {
                mid = (low + high) / 2;

                if (a[mid] == key)
                        return mid;
                else if (a[mid] < key)
                        low = mid + 1;
                else
                        high = mid - 1;
        }
        return -1;
}

// Function to input the array
void inputArray(int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("Enter element for a[%d] : ", i);
        scanf("%d", &a[i]);
    }
}

// Function to print the array
void printArray(int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("Value of a[%d] = %d\n", i, a[i]);
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 64

```
    }
}

// Main function
int main() {
    int a[20], n, key, result;

    printf("Enter value of n : ");
    scanf("%d", &n);

    inputArray(a, n);

    printf("Enter key element : ");
    scanf("%d", &key);

    bubbleSort(a, n);

    printf("After sorting the elements in the array are\n");
    printArray(a, n);

    result = binarySearch(a, n, key);

    if (result != -1) {
        printf("The key element %d is found at the position %d\n", key,
result);
    } else {
        printf("The Key element %d is not found in the array\n", key);
    }

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter value of n :
5
Enter element for a[0] :
4
Enter element for a[1] :

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 65

8
Enter element for a[2] :
6
Enter element for a[3] :
2
Enter element for a[4] :
1
Enter key element :
8
After sorting the elements in the array are
Value of a[0] = 1
Value of a[1] = 2
Value of a[2] = 4
Value of a[3] = 6
Value of a[4] = 8
The key element 8 is found at the position 4

Test Case - 2
User Output
Enter value of n :
7
Enter element for a[0] :
56
Enter element for a[1] :
89
Enter element for a[2] :
63
Enter element for a[3] :
215
Enter element for a[4] :
325
Enter element for a[5] :
156
Enter element for a[6] :
256
Enter key element :
458
After sorting the elements in the array are
Value of a[0] = 56

```
Value of a[1] = 63
Value of a[2] = 89
Value of a[3] = 156
Value of a[4] = 215
Value of a[5] = 256
Value of a[6] = 325
The Key element 458 is not found in the array
```

S.No: 14	Exp. Name: ***Search an Element using Linear Search Process***	**Date: 2026-09-27**

Aim:

Write a C program to search for a key element in a given array using the Linear Search algorithm.

Input Format:

- The program first prompts the user to enter:

```
Enter value of n :
```

- Read an integer n, representing the number of elements in the array.
- For each index i from 0 to $n-1$, the program prompts:

```
Enter element for a[i] :
```

- Read an integer representing the i^{th} element of the array.
- Finally, the program prompts:

```
Enter key element :
```

- Read an integer key, representing the element to be searched in the array.

Output Format:

- If the key element is found, print:

```
The key element <key> is found at the position <pos>
```

- where key is the element being searched and pos is the zero-based index of its first occurrence in the array.
- If the key element is not found, print:

```
The key element <key> is not found in the array
```

- where key is the element being searched.

Note: Do use the **printf()** function with a **newline** character (\n) at the end.

Source Code:

```
Program509.c
```


ID: 250392131492104 3
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
#include<stdio.h>
int main()
{
        int n, a[100], key, i, pos = -1;
        printf("Enter value of n : ");
        scanf("%d", &n);

        for(i = 0;i < n; i++)
        {
                printf("Enter element for a[%d] : ",i);
                scanf("%d", &a[i]);
        }
        printf("Enter key element : ");
        scanf("%d", &key);

        for(i = 0; i < n; i++)
        {
                if(a[i] == key)
                {
                        pos = i;
                        break;
                }
        }
        if(pos != -1)
                printf("The key element %d is found at the position
%d\n", key, pos);
        else
                printf("The key element %d is not found in the array\n",
key);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter value of n :
5
Enter element for a[0] :
45
Enter element for a[1] :

67
Enter element for a[2] :
35
Enter element for a[3] :
28
Enter element for a[4] :
16
Enter key element :
28
The key element 28 is found at the position 3

Test Case - 2
User Output
Enter value of n :
3
Enter element for a[0] :
5
Enter element for a[1] :
7
Enter element for a[2] :
3
Enter key element :
4
The key element 4 is not found in the array

S.No: 15	Exp. Name: ***Hashing implementation***	**Date: 2026-09-27**

ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

Aim:

Write a C program to implement a Hash Table with the following functionalities:

- **insert():** Insert an element into the hash table
- **delete():** Delete an element from the hash table
- **search():** Search for an element in the hash table
- **print():** Display all elements of the hash table
- **exit():** Terminate the program

Menu Options:

```
1.Insert 2.Delete 3.Search 4.Print 5.Exit
```

Option 1: Insert

- Prompt: "Enter an element to be inserted : "
- On successful insertion, print: "Successfully inserted."
- If collision occurs (element already exists at calculated position) print: "Collision Occured."

Option 2: Delete

- Prompt: "Enter an element to be deleted : "
- On successful deletion, print: "Successfully deleted."
- If element not found, print: "Element not found. So cannot delete the element."

Option 3: Search

- Prompt: "Enter an element to be searched : "
- If element found, print: "Element found."
- If element not found, print: "Element not found."

Option 4: Print

- Display all elements in format as shown in below example:

```
[0]=>10 [2]=>22 [3]=>43 ...
```

- Empty slots should not be displayed

Option 5: Exit

- Terminate the program immediately

Note:

- The required menu and print statements are already provided in **HashingMain1.c**.
- Implement the required hash table operations in **HashingOperations.c** only.
- The menu should be displayed repeatedly until the user selects Option 5 (Exit).
- All output must match the format shown in the sample test cases exactly.

Source Code:

HashingMain1.c

```
#include <stdio.h>
#include<stdlib.h>
#define SIZE 10
#include "HashingOperations.c"
int main() {
        int x, op, i = 0;
        for (i = 0; i < SIZE; i++)
                HashTable[i] = -1;
        while (1) {
                printf("1.Insert 2.Delete 3.Search 4.Print 5.Exit\n");
                printf("Enter your option : ");
                scanf("%d", &op);
                switch (op) {
                        case 1: printf("Enter an element to be inserted
: ");
                                        scanf("%d", &x);
                                        insert(x);
                                        break;
                        case 2:
                                        printf("Enter an element to be
deleted : ");
                                        scanf("%d", &x);
                                        delete(x);
                                        break;
                        case 3:
                                        printf("Enter an element to be
searched : ");
                                        scanf("%d", &x);
                                        search(x);
                                        break;
                        case 4:
                                        print();
                                        break;
                        case 5: exit(0);
                }
        }
}
```

HashingOperations.c

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 72

```
int HashTable[SIZE];
int hash(int x) {
        return x % SIZE;
}
void insert(int x) {
                int index = hash(x);
        if(HashTable[index] == -1)
        {
                HashTable[index] = x;
                printf("Successfully inserted.\n");
        }else if (HashTable[index] == x) {
                printf("Collision Occured.\n");
        }
        else
        {
                printf("Collision Occured.\n");
        }
}
void delete(int x) {
        int index = hash(x);

        if(HashTable[index] == x){
                HashTable[index] = -1;
                printf("Successfully deleted.\n");
        }else {
                printf("Element not found. So cannot delete the element.
\n");
        }
}
void search(int x) {
        int index = hash(x);
        if (HashTable[index] == x){
                printf("Element found.\n");
        }
        else {
                printf("Element not found.\n");
        }
}
void print() {
        int i;
        for(i = 0; i < SIZE; i++) {
                if(HashTable[i] != -1) {
                        printf("[%d]=>%d ", i, HashTable[i]);
                }
        }
        printf("\n");
```

```
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
1
Enter an element to be inserted :
10
Successfully inserted.
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
1
Enter an element to be inserted :
22
Successfully inserted.
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
1
Enter an element to be inserted :
43
Successfully inserted.
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
1
Enter an element to be inserted :
44
Successfully inserted.
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
1
Enter an element to be inserted :
55
Successfully inserted.
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :

15
Element not found.
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
2
Enter an element to be deleted :
15
Element not found. So cannot delete the element.
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
2
Enter an element to be deleted :
12
Successfully deleted.
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
4
[1]=>11 [3]=>13 [4]=>14
1.Insert 2.Delete 3.Search 4.Print 5.Exit
Enter your option :
5

S.No: 16	Exp. Name: ***Binary Tree Traversals using Linked List - Recursive***	**Date: 2026-09-27**

Aim:

Write a C program to implement Binary tree traversals using Linked list.
You have to complete the function inorder, preorder and postorder in Traversarls.c where parameters passed are the root reference's of the binary tree **T1.**

Input Format:

- The program prompts the user to enter the values prefixed by "Enter value :" until -1 is entered.

Output Format:

- The first line of the output displays inorder traversal.
- The second line of the output displays preorder traversal.
- The third line of the output displays postorder traversal.

Note:

- Refer to the sample test cases for better understanding regarding input and output formats.

Source Code:

```
TreeMain.c
```


ID: 250392131492104 3
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
// Program for linked implementation of complete binary tree
#include <stdio.h>
#include <stdlib.h>
#include<string.h>
#include<stdbool.h>
#include"TreeStructure.c"
#include"Traversals.c"


// For Queue Size
#define SIZE 100000



// A utility function to create a new Queue
struct Queue* createQueue(int size)
{
    struct Queue* queue = (struct Queue*) malloc(sizeof( struct Queue
));

    queue->front = queue->rear = -1;
    queue->size = size;

    queue->array = (struct node**) malloc
                   (queue->size * sizeof( struct node* ));

    int i;
    for (i = 0; i < size; ++i)
        queue->array[i] = NULL;

    return queue;
}

// Standard Queue Functions
int isEmpty(struct Queue* queue)
{
    return queue->front == -1;
}

int isFull(struct Queue* queue)
{ return queue->rear == queue->size - 1;

}

int hasOnlyOneItem(struct Queue* queue)
{  return queue->front == queue->rear;
```

```
}

void Enqueue(struct node *root, struct Queue* queue)
{
    if (isFull(queue))
        return;
    queue->array[++queue->rear] = root;
    if (isEmpty(queue))
        ++queue->front;
}
struct node* Dequeue(struct Queue* queue)
{
    if (isEmpty(queue))
        return NULL;
    struct node* temp = queue->array[queue->front];

    if (hasOnlyOneItem(queue))
        queue->front = queue->rear = -1;
    else
        ++queue->front;
    return temp;
}

struct node* getFront(struct Queue* queue)
{  return queue->array[queue->front]; }

// A utility function to check if a tree node
// has both left and right children
int hasBothChild(struct node* temp)
{
    return temp && temp->left && temp->right;
}

// Function to insert a new node in complete binary tree
void insert(struct node **root, int data, struct Queue* queue)
{
    // Create a new node for given data
    struct node *temp = newNode(data);
    // If the tree is empty, initialize the root with new node.
    if (!*root)
        *root = temp;
    else
    {
        // get the front node of the queue.
        struct node* front = getFront(queue);
```

```
        // If the left child of this front node doesn’t exist, set the
        // left child as the new node
        if (!front->left)
        {
                front->left = NULL;
            front->left = temp;
        }
        // If the right child of this front node doesn’t exist, set the
        // right child as the new node
        else if (!front->right)
        {
                front->right = NULL;
            front->right = temp;
        }
        // If the front node has both the left child and right child,
        // Dequeue() it.
        if (hasBothChild(front))
        {
                Dequeue(queue);
        }
    }
    // Enqueue() the new node for later insertions
             Enqueue(temp, queue);
}

// Standard level order traversal to test above function
void levelOrder(struct node* root)
{
    struct Queue* queue = createQueue(SIZE);
    Enqueue(root, queue);
    while (!isEmpty(queue))
    {
        struct node* temp = Dequeue(queue);
        printf("%d ", temp->data);
        if (temp->left)
            Enqueue(temp->left, queue);
        if (temp->right)
            Enqueue(temp->right, queue);
    }
}

// Driver program to test above functions
int main()
{
    struct node* T1 = NULL;
    struct Queue* queue1 = createQueue(SIZE);
```

```
    int ele;

    while(1){
       printf("Enter value : ");
       scanf("%d",&ele);
    if(ele==-1){
        break;
    }
    else{
           insert(&T1, ele, queue1);
       }
    }
    inorder(T1);
    printf("\n");
    preorder(T1);
    printf("\n");
    postorder(T1);
}
```

TreeStructure.c

```
// A tree node
struct node
{
    int data;
    struct node *right,*left;
    struct node *root;
};

// A queue node
struct Queue
{
    int front, rear;
    int size;
    struct node* *array;
};

// A utility function to create a new tree node
struct node* newNode(int data)
{
    struct node* temp = (struct node*) malloc(sizeof( struct node ));
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}
```

Traversals.c

```
void inorder( struct node *ptr)
{
        if (ptr == NULL) return;
        inorder(ptr->left);
        printf("%d ",ptr->data);
        inorder(ptr->right);
}

void preorder( struct node *ptr){
        if (ptr == NULL) return;
        printf("%d ",ptr->data);
        preorder(ptr->left);
        preorder(ptr->right);
}

void postorder(struct node *ptr)
{
        if (ptr == NULL) return;
        postorder(ptr->left);
        postorder(ptr->right);
        printf("%d ", ptr->data);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter value :
45
Enter value :
89
Enter value :
56
Enter value :
12
Enter value :
45
Enter value :
26

Enter value :
78
Enter value :
45
Enter value :
-1
45 12 89 45 45 26 56 78
45 89 12 45 45 56 26 78
45 12 45 89 26 78 56 45

Test Case - 2
User Output
Enter value :
15
Enter value :
12
Enter value :
14
Enter value :
18
Enter value :
-1
18 12 15 14
15 12 18 14
18 12 14 15

S.No: 17	Exp. Name: ***Implementation of binary tree using linked list.***	**Date: 2026-09-27**

Aim:

Write a C program to implement Binary tree using Linked list.

Driver code is already provided for you to run the test cases.

Source Code:

```
TreeMain.c
```

Page No: 86
ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
#include <stdio.h>
#include <stdlib.h>
#include"TreeStructure.c"
// For Queue Size
#define SIZE 50
// A utility function to create a new Queue
struct Queue* createQueue(int size) {
        struct Queue* queue = (struct Queue*) malloc(sizeof( struct
Queue ));
        queue->front = queue->rear = -1;
        queue->size = size;
        queue->array = (struct node**) malloc(queue->size * sizeof(
struct node* ));
        int i;
        for (i = 0; i < size; ++i)
                queue->array[i] = NULL;
        return queue;
}
// Standard Queue Functions
int isEmpty(struct Queue* queue) {
        return queue->front == -1;
}
int isFull(struct Queue* queue) {
        return queue->rear == queue->size - 1;
}
int hasOnlyOneItem(struct Queue* queue) {
        return queue->front == queue->rear;
}
void Enqueue(struct node *root, struct Queue* queue) {
        if (isFull(queue))
                return;
        queue->array[++queue->rear] = root;
        if (isEmpty(queue))
                ++queue->front;
}
struct node* Dequeue(struct Queue* queue) {
        if (isEmpty(queue))
                return NULL;
        struct node* temp = queue->array[queue->front];
        if (hasOnlyOneItem(queue))
                queue->front = queue->rear = -1;
        else
                ++queue->front;
        return temp;
}
struct node* getFront(struct Queue* queue) {
```

```
        return queue->array[queue->front];
}
int hasBothChild(struct node* temp) {
        return temp && temp->left && temp->right;
}
void insert(struct node **root, int data, struct Queue* queue) {
        struct node* newNode = (struct node*) malloc(sizeof(struct
node));
        newNode->data = data;
        newNode->left  = newNode->right = NULL;
        if (*root == NULL) {
                *root = newNode;
                return;
        }
        Enqueue(*root, queue);
        while (!isEmpty(queue)) {
                struct node* temp = Dequeue(queue);

                if (temp->left == NULL) {
                        temp->left = newNode;
                        queue->front = queue->rear = -1;
                        return;
                }else {
                        Enqueue(temp->left, queue);
                }
                if (temp->right == NULL) {
                        temp->right = newNode;
                        queue->front = queue->rear = -1;
                        return;
                }else {
                        Enqueue(temp->right, queue);
                }
        }
}
void removethedeepestnode(struct node *root, struct node *d_node){
    struct Queue* queue = createQueue(SIZE);
        Enqueue(root,queue);

        while (!isEmpty(queue)) {
                struct node* temp = Dequeue(queue);
                if (temp->left) {
                        if (temp->left == d_node) {
                                temp->left = NULL;
                                free(d_node);
                                return;
                        }else {
```

```
                                Enqueue(temp->left, queue);
                        }
                }
                if (temp->right) {
                        if (temp->right == d_node) {
                                temp->right = NULL;
                                free(d_node);
                                return;
                        }else {
                                Enqueue(temp->right, queue);
                        }
                }
        }
}
void levelOrder(struct node* root) {
        struct Queue* queue = createQueue(SIZE);
        Enqueue(root, queue);
        while (!isEmpty(queue)) {
                struct node* temp = Dequeue(queue);
                printf("%d ", temp->data);
                if (temp->left)
                        Enqueue(temp->left, queue);
                if (temp->right)
                        Enqueue(temp->right, queue);
        }
}

void findDeepestnode(struct node *root, struct node *search){
    struct Queue* queue = createQueue(SIZE);
        Enqueue(root, queue);
        struct node *temp, *par;
        while (!isEmpty(queue))
        {   par = temp;
                temp = Dequeue(queue);
                if (temp->left)
                        Enqueue(temp->left, queue);
                if (temp->right)
                        Enqueue(temp->right, queue);
        }
        search->data = temp->data;
    removethedeepestnode(root,temp);
}
struct node* searchingNode( struct node *root, int data) {
     struct Queue* queue = createQueue(SIZE);
        Enqueue(root, queue);
        struct node *searchNode = NULL;
```

```
        while (!isEmpty(queue))
        {       struct node* temp = Dequeue(queue);
                if(temp->data == data){
                    searchNode = temp;
                    return searchNode;
                }
                if (temp->left)
                        Enqueue(temp->left, queue);
                if (temp->right)
                        Enqueue(temp->right, queue);
        }
   return searchNode;
}
// Driver program to test above functions
int main()
{       struct node* root = NULL;
        struct Queue* queue = createQueue(SIZE);
        int i;
        int limit;
        int data1;
        printf("Enter the limit to form binary tree: ");
        scanf("%d",&limit);
        for(i = 1; i <= limit; ++i) {
                printf("Enter node: ");
                scanf("%d",&data1);
                insert(&root, data1, queue);
        }
        printf("Level order traversal before deletion\n");
        levelOrder(root);
        int data;
        printf("\nEnter data to delete: ");
        scanf("%d",&data);
        struct node *search = searchingNode(root, data);
        if(search == NULL)
            printf("Node is not present\n");
        else{
            findDeepestnode(root, search);
            printf("Level order traversal after deletion\n");
            levelOrder(root);
        }
        return 0;
}
```

TreeStructure.c

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 90

```
// A tree node
struct node
{       int data;
        struct node *right,*left;
};

// A queue node
struct Queue
{       int front, rear;
        int size;
        struct node* *array;
};

// A utility function to create a new tree node
struct node* newNode(int data)
{       struct node* temp = (struct node*) malloc(sizeof( struct node
));
        temp->data = data;
        temp->left = temp->right = NULL;
        return temp;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the limit to form binary tree:
5
Enter node:
45
Enter node:
16
Enter node:
26
Enter node:
36
Enter node:
78
Level order traversal before deletion
45 16 26 36 78

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 91

Enter data to delete:
26
Level order traversal after deletion
45 16 78 36

Test Case - 2
User Output
Enter the limit to form binary tree:
6
Enter node:
45
Enter node:
15
Enter node:
26
Enter node:
48
Enter node:
26
Enter node:
36
Level order traversal before deletion
45 15 26 48 26 36
Enter data to delete:
86
Node is not present

S.No: 18	Exp. Name: ***Program to insert into BST and traversal using In-order, Pre-order and Post-order***	**Date: 2026-09-27**

Aim:

Write a program to create a binary search tree of integers and perform the following operations using linked list.

1. Insert a node
2. In-order traversal
3. Pre-order traversal
4. Post-order traversal

Note: Write the code in **InsertAndTraversals.c** file.

Source Code:

```
BinarySearchTree.c
```


ID: 250392131492104 3
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
#include<stdio.h>
#include<stdlib.h>
#include "InsertAndTraversals.c"

void main() {
        int x, op;
        BSTNODE root = NULL;
        while(1) {
                printf("1.Insert 2.Inorder Traversal 3.Preorder
Traversal 4.Postorder Traversal 5.Exit\n");
                printf("Enter your option : ");
                scanf("%d", &op);
                switch(op) {
                        case 1: printf("Enter an element to be inserted
: ");
                                        scanf("%d", &x);
                                        root = insertNodeInBST(root,x);
                                        break;
                        case 2:
                                        if(root == NULL) {
                                                printf("Binary Search
Tree is empty.\n");
                                        }
                                        else {
                                                printf("Elements of the
BST (in-order traversal): ");
                                                inorderInBST(root);
                                                printf("\n");
                                                }
                                        break;
                        case 3:
                                        if(root == NULL) {
                                                printf("Binary Search
Tree is empty.\n");
                                        }
                                        else {
                                                printf("Elements of the
BST (pre-order traversal): ");
                                                preorderInBST(root);
                                                printf("\n");
                                                }
                                        break;
                        case 4:
                                        if(root == NULL) {
                                                printf("Binary Search
Tree is empty.\n");
```

```
                                        }
                                        else {
                                                printf("Elements of the
BST (post-order traversal): ");
                                                postorderInBST(root);
                                                printf("\n");
                                                }
                                        break;
                        case 5:
                                        exit(0);
                }
        }
}
```

InsertAndTraversals.c

```
struct node {
        int data;
        struct node *left, *right;
};

typedef struct node *BSTNODE;

BSTNODE newNodeInBST(int item) {
        BSTNODE temp =  (BSTNODE)malloc(sizeof(struct node));
        temp->data = item;
        temp->left = temp->right = NULL;
        return temp;
}

BSTNODE insertNodeInBST(BSTNODE node, int ele) {
        if (node == NULL) {
                printf("Successfully inserted.\n");
                return newNodeInBST(ele);
        }
        if (ele < node->data) {
                node->left = insertNodeInBST(node->left, ele);
        }
        else if (ele > node->data) {
                node->right = insertNodeInBST(node->right, ele);
        }
        else {
                printf("Element already exists in BST.\n");
        }
        return node;
}

void inorderInBST(BSTNODE root) {
        if (root != NULL) {
                inorderInBST(root->left);
                printf("%d ", root->data);
                inorderInBST(root->right);
        }
}

void preorderInBST(BSTNODE root) {
        if (root != NULL) {
                printf("%d ", root->data);
                preorderInBST(root->left);
                preorderInBST(root->right);
        }
```

```
}

void postorderInBST(BSTNODE root) {
        if (root != NULL) {
                postorderInBST(root->left);
                postorderInBST(root->right);
                printf("%d ", root->data);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
54
Successfully inserted.
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
28
Successfully inserted.
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
62
Successfully inserted.
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
2

Elements of the BST (in-order traversal): 28 54 62
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
3
Elements of the BST (pre-order traversal): 54 28 62
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
4
Elements of the BST (post-order traversal): 28 62 54
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
5

Test Case - 2
User Output
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
100
Successfully inserted.
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
20
Successfully inserted.
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
200
Successfully inserted.

Elements of the BST (post-order traversal): 10 30 20 150 300 200 100
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
5

Test Case - 3
User Output
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
12
Successfully inserted.
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
12
Element already exists in BST.
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
2
Elements of the BST (in-order traversal): 12
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
3
Elements of the BST (pre-order traversal): 12
1.Insert 2.Inorder Traversal 3.Preorder Traversal 4.Postorder Traversal 5.Exit
Enter your option :
5

S.No: 19	Exp. Name: ***Count the number of nodes in Binary Search Tree.***	**Date: 2026-09-27**

Aim:

Write a C program which counts the number of nodes in Binary Search Tree.

Source Code:

Nodes.c

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 101

```
#include<stdlib.h>
#include<stdio.h>

struct binarytree
{   int data;
    struct binarytree * right, * left;
};

typedef struct binarytree node;

void insert(node ** tree, int val)
{
        node *temp = NULL;
        if (*tree == NULL)
        {
                temp = (node *)malloc(sizeof(node));

                temp->data = val;
                temp->left = NULL;
                temp->right = NULL;
                *tree = temp;
                return;
        }
        if (val < (*tree)->data)
        {
                insert(&((*tree)->left), val);
        }
        else if (val > (*tree)->data)
        {
                insert(&((*tree)->right), val);
        }
        else
        {
                printf("Duplicate node\n");
        }
}

int count(node *tree)
{
        if (tree == NULL)
        {
                return 0;
        }
        return 1 + count(tree->left) + count(tree->right);
}
```

```
int main()
{   node *root, *tmp;
    int c, ele, op;

    root = NULL;

    while(1)
    {   printf("1.Insertion\n2.Count\n3.Exit\n");
        scanf("%d",&op);
        switch(op)
        {   case 1:
                printf("Enter element : ");
                scanf("%d",&ele);
                insert(&root, ele);
                break;
            case 2:
                c = count(root);
                printf("Number of nodes %d \n",c);
                break;
            case 3:
                exit(0);
            default:
                printf("Out of Range\n");
        }
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1.Insertion
2.Count
3.Exit
1
Enter element :
1
1.Insertion
2.Count
3.Exit
1

Enter element :
2
1.Insertion
2.Count
3.Exit
1
Enter element :
3
1.Insertion
2.Count
3.Exit
1
Enter element :
4
1.Insertion
2.Count
3.Exit
1
Enter element :
5
1.Insertion
2.Count
3.Exit
2
Number of nodes 5
1.Insertion
2.Count
3.Exit
3

Test Case - 2
User Output
1.Insertion
2.Count
3.Exit
1
Enter element :
15
1.Insertion
2.Count

1
Enter element :
15
Duplicate node
1.Insertion
2.Count
3.Exit
3

Test Case - 3
User Output
1.Insertion
2.Count
3.Exit
45
Out of Range
1.Insertion
2.Count
3.Exit
3

S.No: 20	Exp. Name: ***Min Heap Operations***	**Date: 2026-09-28**

Aim:

Write a C program that creates a Min Heap of integers, displays the heap, and performs the deletion (extract-min) operation.

Input Format:

- The program first prompts the user to enter:

`Enter no of elements:`

- Read an integer n, representing the number of elements to be inserted into the Min Heap.
- The program then prompts:

`Enter elements:`

- Read n space-separated integers representing the elements to be inserted into the Min Heap.

Output Format:

- Print the Min Heap after all insertions in the following format:

`Min Heap: <heap_elements>`

- where <heap_elements> are the elements of the Min Heap printed in level-order, separated by spaces.
- Then print:

`Extracting elements from the min heap:`

- For each extraction, print:

`Extracted: <element>`

- where <element> is the minimum element extracted from the Min Heap.
- Each extracted element should be printed on a new line until the heap becomes empty.

Source Code:

minHeapOfIntegers.c

```c
#include <stdio.h>

#define MAX_HEAP_SIZE 100

// Function to swap two integers
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

// Function to perform min heapify on a subtree rooted at index i
void minHeapify(int arr[], int n, int i) {
        int smallest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] < arr[smallest])
                smallest = left;
        if (right < n && arr[right] < arr[smallest])
                smallest = right;
        if (smallest != i) {
                swap(&arr[i], &arr[smallest]);
                minHeapify(arr, n, smallest);
        }
}

// Function to build a min heap
void buildMinHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        minHeapify(arr, n, i);
}

// Function to display the min heap
void displayHeap(int arr[], int n) {
        printf("Min Heap: ");
        for (int i = 0; i < n; i++)
                printf("%d ", arr[i]);
        printf("\n");

}

int extractMin(int arr[], int *n) {
        if(*n <= 0)
                return -1;
        int minElement = arr[0];
```

```
        arr[0] = arr[*n - 1];
        (*n)--;

        if (*n > 0)
                minHeapify(arr, *n, 0);
        return minElement;
}



int main() {
    int arr[MAX_HEAP_SIZE];
    int n;

    printf("Enter no of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    buildMinHeap(arr, n);
    displayHeap(arr, n);

    printf("Extracting elements from the min heap:\n");
    while (n > 0) {
        int minElement = extractMin(arr, &n);
        if (minElement == -1)
            printf("Heap is empty.\n");
        else
            printf("Extracted: %d\n", minElement);
    }

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter no of elements:
5

Enter elements:
1 2 6 4 8
Min Heap: 1 2 6 4 8
Extracting elements from the min heap:
Extracted: 1
Extracted: 2
Extracted: 4
Extracted: 6
Extracted: 8

Test Case - 2
User Output
Enter no of elements:
6
Enter elements:
2 5 1 6 8 4
Min Heap: 1 5 2 6 8 4
Extracting elements from the min heap:
Extracted: 1
Extracted: 2
Extracted: 4
Extracted: 5
Extracted: 6
Extracted: 8

S.No: 21	Exp. Name: ***Binary Heap***	**Date:** **2026-09-28**

Aim:

Write a C program to implement Binary Heap operations.

Source Code:

```
binaryheap.c
```

```
#include<stdio.h>
#include<stdlib.h>

int heap[100];
int size = 0;
int capacity;

void insert(int value)
{
        if (size >= capacity)
        {
                return;
        }
        int i = size;
        heap[size] = value;
        size++;

        while (i > 0)
                {
                        int parent = (i - 1) / 2;

                        if (heap[parent] <= heap[i])
                                break;

                        int temp = heap[parent];
                        heap[parent] = heap[i];
                        heap[i] = temp;

                        i = parent;
                }
}

void display()
{
        printf("Heap: ");

        for (int i = 0; i < size; i++)
                {
                        printf("%d", heap[i]);

                        if (i < size)
                                printf(" ");
                }
        printf("\n");
}
int main()
```

```
{
        printf("Enter the capacity of the Binary Heap: ");
        scanf("%d", &capacity);
        printf("Enter elements to add to the heap (-1 to stop):");

        int value;
        while(1)
                {
                        scanf("%d", &value);

                        if (value == -1)
                                break;
                        insert(value);
                }
        printf("\nHeap after insertions:\n");
        display();

        if(size > 0)
                printf("Min value is %d\n", heap[0]);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the capacity of the Binary Heap:
10
Enter elements to add to the heap (-1 to stop):
10 5 20 3 8 15 30 -1
Heap after insertions:
Heap: 3 5 15 10 8 20 30
Min value is 3

Test Case - 2
User Output
Enter the capacity of the Binary Heap:
8
Enter elements to add to the heap (-1 to stop):

4 1 2 6 7 3 8 5 -1
Heap after insertions:
Heap: 1 4 2 5 7 3 8 6
Min value is 1

S.No: 22	Exp. Name: ***Max Heap of Integers***	**Date: 2026-09-28**

Aim:

Write a C program that creates a Max Heap of integers, displays the heap, and performs the deletion (extract-max) operation.

Input Format:

- The program first prompts the user to enter:

`Enter no of elements:`

- Read an integer n, representing the number of elements to be inserted into the Max Heap.
- The program then prompts:

`elements:`

- Read n space-separated integers representing the elements to be inserted into the Max Heap.

Output Format:

- Print the Max Heap after all insertions in the following format:

`Max Heap: <heap_elements>`

- where <heap_elements> are the elements of the Max Heap printed in level-order, separated by spaces.
- Then print:

`Extracting elements from max heap:`

- For each extraction, print:

`Extracted: <element>`

- where <element> is the maximum element extracted from the Max Heap.
- Each extracted element should be printed on a new line until the heap becomes empty. The extracted elements will be printed in descending order.

Source Code:

maxHeapOfIntegers.c

```c
#include <stdio.h>

#define MAX_HEAP_SIZE 100

// Function to swap two integers
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

// Function to perform max heapify on a subtree rooted at index i
void maxHeapify(int arr[], int n, int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
                largest = left;
        if (right < n && arr[right] > arr[largest])
                largest = right;
        if (largest !=i)  {
                swap(&arr[i], &arr[largest]);
                maxHeapify(arr, n, largest);
        }
}
void buildMaxHeap(int arr[],int n) {
        for (int i = n / 2 -1; i >= 0; i--) {
                maxHeapify(arr, n, i);
        }
}
void displayHeap(int arr[], int n) {
        printf("Max Heap: ");
        for (int i = 0; i < n; i++) {
                printf("%d", arr[i]);
                if (i < n)
                        printf(" ");
        }
        printf("\n");
}

int extractMax(int arr[], int *n) {
        if (*n <= 0)
                return -1;
        int maxElement = arr[0];
        arr[0] = arr[*n - 1];
```

```
        (*n)--;

        if (*n > 0)
                maxHeapify(arr, *n, 0);
        return maxElement;

}

int main() {
    int arr[MAX_HEAP_SIZE];
    int n;

    printf("Enter no of elements: ");
    scanf("%d", &n);

    printf("elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    buildMaxHeap(arr, n);
    displayHeap(arr, n);

    printf("Extracting elements from max heap:\n");
    while (n > 0) {
        int maxElement = extractMax(arr, &n);
        if (maxElement == -1)
            printf("Heap is empty.\n");
        else
            printf("Extracted: %d\n", maxElement);
    }

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter no of elements:
5
elements:
6 4 5 8 2

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 117

```
Max Heap: 8 6 5 4 2
Extracting elements from max heap:
Extracted: 8
Extracted: 6
Extracted: 5
Extracted: 4
Extracted: 2
```

S.No: 23	Exp. Name: ***Implementation of Depth First Search***	**Date: 2026-09-29**

Aim:

Implement a program to perform **Depth-First Search** (DFS) traversal on a directed graph using adjacency lists.

- The graph is represented using an adjacency list.
- Vertices are numbered from 0 or 1 up to n-1 or n, depending on input.
- After building the graph with the given number of vertices and edges, perform DFS traversal starting from a given start vertex.
- Print each visited vertex in the order they are visited by the DFS.

Input Format:

- **First line:** An integer$n$ — the number of vertices in the graph.
- **Second line:** An integer E — the number of edges.
- **Next E Lines:** For each of the next E edges which denotes a directed edge from vertex $source$ to vertex $destination$:
  - First, an integer $source$ on one line.
  - Then, an integer $destination$ on the next line.
- **Last line:** An integer v — the start vertex for DFS traversal.

Output Format:

- Print the line "DFS of graph:"
- Then print each vertex of the graph as it is visited by the DFS algorithm, each on a new line.

Note:

- The vertices can be numbered starting from either 0 or 1, as per the input.

Source Code:

GraphDFS.c

ID: 2503921314921043 Page No: 119
2025-2029-CSE(CS) PSNA College of Engineering and Technology

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    struct node *next;
    int vertex;
};

typedef struct node *GNODE;

GNODE graph[20];
int visited[20];
int n;
void DFS(int i)
{
        GNODE p;

        printf("\n%d", i);
        p = graph[i];
        visited[i] = 1;

        while(p != NULL)
                {
                        i = p->vertex;
                        if(!visited[i])
                                DFS(i);

                        p = p->next;
                }
}
void main() {
    int n,E,i,s,d,v;
    GNODE q,p;
    printf("Enter no of vertices: ");
        scanf("%d",&n);
        printf("Enter no of edges: ");
        scanf("%d",&E);
        for(i=1;i<=E;i++) {
                printf("Enter source: ");
                scanf("%d",&s);
                printf("Enter destination: ");
                scanf("%d",&d);
                q=(GNODE)malloc(sizeof(struct node));
        q->vertex=d;
        q->next=NULL;
        if(graph[s]==NULL)
```

```
                graph[s]=q;
        else {
                p=graph[s];
                while(p->next!=NULL)
                p=p->next;
                p->next=q;
        }
        }
    for(i=0;i<n;i++)
        visited[i]=0;
    printf("Enter Start Vertex for DFS: ");
    scanf("%d", &v);
    printf("DFS of graph: ");
    DFS(v);
    printf("\n");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter no of vertices:
5
Enter no of edges:
5
Enter source:
1
Enter destination:
2
Enter source:
1
Enter destination:
4
Enter source:
4
Enter destination:
2
Enter source:
2

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 121

Enter destination:
5
Enter source:
2
Enter destination:
5
Enter source:
3
Enter destination:
6
Enter source:
4
Enter destination:
5
Enter source:
7
Enter destination:
6
Enter source:
7
Enter destination:
4
Enter Start Vertex for DFS:
6
DFS of graph:
6

S.No: 24	Exp. Name: ***Implementation of Breadth First Search***	**Date: 2026-10-19**

Aim:

Implement a program to perform **Breadth-First Search** (BFS) traversal on a directed graph using adjacency lists.

- The graph is represented using adjacency lists.
- Vertices can be numbered starting from 0 or 1, as per the input.
- After constructing the graph with the given number of vertices and edges, perform BFS traversal starting from a given start vertex.
- Print each vertex as it is visited during the BFS traversal.

Input Format:

- The first line contains an integer n - the number of vertices in the graph.
- The second line contains an integer E - the number of edges.
- Next E Lines: For each of the next E edges which denotes a directed edge from vertex $source$ to vertex $destination$:
    - First, an integer $source$ on one line.Then, an integer $destination$ on the next line.
- The last line contains an integer v - the start vertex for BFS traversal.

Output Format:

- Print the line: "BFS of graph: "
- Then print each visited vertex during BFS on a new line in the order they are visited.

Note:

- Vertices are numbered according to the input and your implementation should support them as given.

Source Code:

```
GraphsBFS.c
```

ID: 2503921314921043 Page No: 124
2025-2029-CSE(CS) PSNA College of Engineering and Technology

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 99

struct node {
    struct node *next;
    int vertex;
};

typedef struct node *GNODE;
int visited[20];
GNODE graph[20];

void BFS(int v)
{
        GNODE p;
        int queue[20];
        int front = 0, rear = 0;

        queue[rear++] = v;
        visited[v] = 1;
        printf("\n%d",v);
        while (front < rear)
                {
                        v = queue[front++];
                        p=graph[v];

                        while ( p != NULL)
                                {
                                        if (visited[p->vertex] == 0)
                                        {
                                                queue[rear++] = p-
>vertex;

                                                visited[p->vertex] = 1;
                                                printf("\n%d",p-
>vertex);

                                        }
                                        p = p->next;
                                }
                }
}

int main() {
    int n, E, s, d, v, i;
    GNODE p, q;
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 125

```
    // Initialize graph to NULL
    for (i = 0; i < 20; i++)
        graph[i] = NULL;

    printf("Enter no of vertices: ");
    scanf("%d", &n);

    printf("Enter no of edges: ");
    scanf("%d", &E);

    for (i = 1; i <= E; i++) {
        printf("Enter source: ");
        scanf("%d", &s);
        printf("Enter destination: ");
        scanf("%d", &d);

        q = (GNODE)malloc(sizeof(struct node));
        q->vertex = d;
        q->next = NULL;

        if (graph[s] == NULL)
            graph[s] = q;
        else {
            p = graph[s];
            while (p->next != NULL)
                p = p->next;
            p->next = q;
        }
    }

    for (i = 1; i <= n; i++)
        visited[i] = 0;

    printf("Enter Start Vertex for BFS: ");
    scanf("%d", &v);

    printf("BFS of graph: ");
    BFS(v);
    printf("\n");

    return 0;
}
```

Execution Results - All test cases have succeeded!

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 126

Test Case - 1
User Output
Enter no of vertices:
5
Enter no of edges:
5
Enter source:
1
Enter destination:
2
Enter source:
1
Enter destination:
4
Enter source:
4
Enter destination:
2
Enter source:
2
Enter destination:
3
Enter source:
4
Enter destination:
5
Enter Start Vertex for BFS:
1
BFS of graph:
1
2
4
3
5

Test Case - 2
User Output
Enter no of vertices:

4
Enter no of edges:
3
Enter source:
1
Enter destination:
2
Enter source:
2
Enter destination:
3
Enter source:
3
Enter destination:
4
Enter Start Vertex for BFS:
2
BFS of graph:
2
3
4

S.No: 25	Exp. Name: ***Dijkstra's Shortest Path Algorithm***	**Date: 2026-10-19**

Aim:

Given a graph G and source vertex S, Dijkstra's shortest path algorithm is used to find the shortest paths from source S to all vertices in the given graph.

The Dijkstra algorithm is also known as the single-source shortest path algorithm. It is based on the greedy technique. A little variation in the algorithm can find the shortest path from the source nodes to all the other nodes in the graph.

The function **void dijkstra(int** $G[MAX][MAX]$**, int** n**, int** $startnode$**)** computes and prints the shortest path distances and corresponding paths from the given source node to all other nodes in a weighted directed graph using Dijkstra's algorithm. It outputs the distance or "INF" if unreachable, along with the path or "NO PATH" for each node.

Note:

- Vertices are numbered from 1 through V.
- All input values are separated by spaces and/or newlines.

Sample Input and Output:

```
Enter the number of vertices : 4
Enter the number of edges : 5
Enter source : 1
Enter destination : 2
Enter weight : 4
Enter source : 1
Enter destination : 4
Enter weight : 10
Enter source : 1
Enter destination : 3
Enter weight : 6
Enter source : 2
Enter destination : 4
Enter weight : 5
Enter source : 3
Enter destination : 4
Enter weight : 2
Enter the source :1
Node    Distance    Path
2   4   2<-1
3   6   3<-1
4   8   4<-3<-1
```

Source Code:

Dijkstras.c

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 129

```
#include <limits.h>
#include <stdio.h>
#define MAX 20
int V, E;
int graph[MAX][MAX];
#define INFINITY 99999

void dijkstra(int G[MAX][MAX], int n, int startnode) {
        int dist[MAX],visited[MAX]={0},parent[MAX];
        int i,j,u,min;
        for(i=1;i<=n;i++){
                dist[i]=(G[startnode][i] !=0 ) ? G[startnode][i] :
INFINITY;
                parent[i] = startnode;
        }
        dist[startnode] = 0;
        visited[startnode] = 1;
        for(i=1;i<=n;i++){
                min = INFINITY;
                u=-1;
                for(j=1;j<=n;j++)
                        if(!visited[j] && dist[j] < min){
                                min = dist[j];
                                u=j;
                        }
                if(u==-1) break;
                visited[u]=1;
                for(j=1;j<=n;j++)
                        if(!visited[j] && G[u][j] > 0 && dist[u] + G[u]
[j]<dist[j]){
                                dist[j]=dist[u] + G[u][j];
                                parent[j] = u;
                        }
        }
        printf("Node\tDistance\tPath\n");
        for(i=1;i<=n;i++){
                if(i==startnode) continue;
                if(dist[i] == INFINITY){
                        printf("   %d\t     INF\tNO PATH\n",i);
                        continue;
                }
                printf("   %d\t       %d\t",i,dist[i]);
                j=i;
                for(int k=0;k<n && j!=startnode && j!=-1;k++){
                        printf("%d<-",j);
                        j=parent[j];
```

```
                }
                if(j==startnode)
                        printf("%d\n",startnode);
                else
                        printf("NO PATH\n");
;       }

}
int main() {
        int s, d, w, i, j;
        printf("Enter the number of vertices : ");
        scanf("%d", &V);
        printf("Enter the number of edges : ");
        scanf("%d", &E);
        for(i = 1 ; i <= V; i++) {
                for(j = 1; j <= V; j++) {
                        graph[i][i] = 0;
                }
        }
        for(i = 1; i <= E; i++) {
                printf("Enter source : ");
                scanf("%d", &s);
                printf("Enter destination : ");
                scanf("%d", &d);
                printf("Enter weight : ");
                scanf("%d", &w);
                if(s > V || d > V || s <= 0 || d <= 0) {
                        printf("Invalid index. Try again.\n");
                        i--;
                        continue;
                } else {
                        graph[s][d] = w;
                }
        }
        printf("Enter the source :");
        scanf("%d", &s);
        dijkstra(graph, V, s);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 131

User Output

```
Enter the number of vertices :
4
Enter the number of edges :
5
Enter source :
1
Enter destination :
2
Enter weight :
4
Enter source :
1
Enter destination :
4
Enter weight :
10
Enter source :
1
Enter destination :
3
Enter weight :
6
Enter source :
2
Enter destination :
4
Enter weight :
5
Enter source :
3
Enter destination :
4
Enter weight :
2
Enter the source :
1
Node    Distance        Path
   2           4        2<-1
   3           6        3<-1
```

4	8	4<-3<-1

Test Case - 2
User Output
Enter the number of vertices :
5
Enter the number of edges :
6
Enter source :
1
Enter destination :
2
Enter weight :
2
Enter source :
1
Enter destination :
5
Enter weight :
3
Enter source :
2
Enter destination :
4
Enter weight :
4
Enter source :
2
Enter destination :
3
Enter weight :
7
Enter source :
4
Enter destination :
3
Enter weight :
2
Enter source :

```
Enter destination :
2
Enter weight :
3
Enter source :
4
Enter destination :
3
Enter weight :
8
Enter the source :
1
Node    Distance        Path
   2           4        2<-1
   3         INF        NO PATH
   4         INF        NO PATH
```

S.No: 26	Exp. Name: ***Kruskal's Algorithm***	**Date: 2026-10-19**

Aim:

Write a C program for constructing a minimum cost spanning tree of a graph using Kruskal's Algorithm.

Note: The main function is already provided; complete the kruskal() function to implement Kruskal's Algorithm.

Source Code:

```
KruskalSpanningTree.c
```

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

int i, j, k, a, b, u, v, n, e, s, d, w, ne = 1;
int min, mincost = 0, cost[9][9], parent[9];

int find(int i) {
        while(parent[i])
                i = parent[i];
        return i;
}

int uni(int i, int j) {
        if(i != j) {
                parent[j] = i;
                return 1;
        }
        return 0;
}

void kruskal() {
        int i,j,k,a,b,u,v;
        mincost=0;
        for(i=1;i<=n;i++)
                parent[i]=0;
        for(k=1;k<n;){
                min = 999;
                for(i=1;i<=n;i++){
                        for(j=1;j<=n;j++){
                                if(cost[i][j]<min){
                                        min = cost[i][j];
                                        a=u=i;
                                        b=v=j;
                                }
                        }
                }
                u=find(u);
                v=find(v);
                if(u!=v){
                        uni(u,v);
                        printf("Edge cost from %d to %d :
%d\n",a,b,min);
                        mincost+=min;
                        k++;
                }
```

```
                cost[a][b]=cost[b][a]=999;
        }
        printf("Minimum cost of spanning tree = %d\n",mincost);
}

void main() {
        printf("Enter the number of vertices : ");
        scanf("%d", &n);
        printf("Enter the number of edges : ");
        scanf("%d", &e);
        for(i = 1; i <= e; i++) {
                printf("Enter source : ");
                scanf("%d", &s);
                printf("Enter destination : ");
                scanf("%d", &d);
                printf("Enter weight : ");
                scanf("%d", &w);
                if(s <= 0 || d <= 0 || s > n || d > n || w < 0 ) {
                        printf("Invalid data.Try again.\n");
                        i--;
                        continue;
                }
                cost[s][d] = w;
        }
        for(i = 1; i <= n; i++) {
                for(j = 1; j <= n; j++) {
                        if(cost[i][j] == 0)
                                cost[i][j] = 999;
                }
        }
        printf("The edges of Minimum Cost Spanning Tree are : \n");
        kruskal();
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the number of vertices :
7
Enter the number of edges :
11

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 138

```
Enter source :
1
Enter destination :
2
Enter weight :
7
Enter source :
2
Enter destination :
3
Enter weight :
8
Enter source :
1
Enter destination :
4
Enter weight :
5
Enter source :
2
Enter destination :
4
Enter weight :
9
Enter source :
2
Enter destination :
5
Enter weight :
7
Enter source :
3
Enter destination :
5
Enter weight :
5
Enter source :
4
Enter destination :
5
```

```
Enter the number of edges :
2
Enter source :
0
Enter destination :
1
Enter weight :
5
Invalid data.Try again.
Enter source :
1
Enter destination :
2
Enter weight :
5
Enter source :
2
Enter destination :
3
Enter weight :
6
The edges of Minimum Cost Spanning Tree are :
Edge cost from 1 to 2 : 5
Edge cost from 2 to 3 : 6
Minimum cost of spanning tree = 11
```

Source Code:

PrimsMinimumSpanningTree.c

```
#include<stdio.h>
#include<conio.h>

// complete the missing code..
int j,i,n,e,s,d,w,cost[9][9];
void prims() {
        int i,j,u,v,min,mincost=0,ne=1,a,b;
        int visited[9] = {0};
        visited[1] = 1;
        while(ne<n){
                min = 999;
                for(i=1;i<=n;i++){
                        for(j=1;j<=n;j++){
                                if(cost[i][j] < min && visited[i] && !
visited[j]){
                                        min = cost[i][j];
                                        u=i;
                                        v=j;
                                }
                        }
                }
                if(visited[u] && !visited[v]){
                        printf("Edge cost from %d to %d :
%d\n",u,v,min);
                        mincost += min;
                        visited[v]=1;
                        ne++;
                }
                cost[u][v]=cost[v][u] = 999;
        }

        printf("Minimum cost of spanning tree = %d\n",mincost);

}
void main() {
        printf("Enter the number of vertices : ");
        scanf("%d", &n);
        printf("Enter the number of edges : ");
        scanf("%d", &e);
        for(i=1; i<=e; i++) {
                printf("Enter source : ");
                scanf("%d", &s);
                printf("Enter destination : ");
                scanf("%d", &d);
                printf("Enter weight : ");
                scanf("%d", &w);
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 144

```
                if(s <= 0 || d <= 0 || s > n || d > n || w < 0 ) {
                        printf("Invalid data.Try again.\n");
                        i--;
                        continue;
                }
                cost[s][d] = w;
                cost[d][s] = w;
        }
        for(i=1; i <= n; i++) {
            for(j=1; j<=n; j++) {
                        if(cost[i][j] == 0)
                            cost[i][j] = 999;
            }
        }
        printf("The edges of Minimum Cost Spanning Tree are : \n");
        prims();
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the number of vertices :
7
Enter the number of edges :
11
Enter source :
1
Enter destination :
2
Enter weight :
7
Enter source :
2
Enter destination :
3
Enter weight :
8
Enter source :
1

Enter source :
5
Enter destination :
7
Enter weight :
9
Enter source :
6
Enter destination :
7
Enter weight :
11
The edges of Minimum Cost Spanning Tree are :
Edge cost from 1 to 4 : 5
Edge cost from 4 to 6 : 6
Edge cost from 1 to 2 : 7
Edge cost from 2 to 5 : 7
Edge cost from 5 to 3 : 5
Edge cost from 5 to 7 : 9
Minimum cost of spanning tree = 39

Test Case - 2
User Output
Enter the number of vertices :
7
Enter the number of edges :
9
Enter source :
2
Enter destination :
7
Enter weight :
14
Enter source :
1
Enter destination :
2
Enter weight :
28

Enter weight :
24
The edges of Minimum Cost Spanning Tree are :
Edge cost from 1 to 6 : 10
Edge cost from 6 to 5 : 25
Edge cost from 5 to 4 : 22
Edge cost from 4 to 3 : 12
Edge cost from 3 to 2 : 16
Edge cost from 2 to 7 : 14
Minimum cost of spanning tree = 99

Test Case - 3
User Output
Enter the number of vertices :
2
Enter the number of edges :
1
Enter source :
3
Enter destination :
1
Enter weight :
5
Invalid data.Try again.
Enter source :
0
Enter destination :
1
Enter weight :
5
Invalid data.Try again.
Enter source :
1
Enter destination :
2
Enter weight :
5
The edges of Minimum Cost Spanning Tree are :
Edge cost from 1 to 2 : 5
Minimum cost of spanning tree = 5